

MLP vs. Traditional Baselines for Spray Penetration

Mie Spray Postprocessing Project

2026 年 5 月 12 日

Scope of This Overview

Role

This short deck is the bridge between method families. The detailed evidence is split into three thesis decks: baseline comparison, engineered-feature scaling, and probabilistic calibration.

Current headline

On the common external clean diagnostic set (795 561 points, 33 845 trajectories), the $\Delta P^{0.5}$ Stage-3 MLP five-seed mean beats calibrated Hiroyasu–Arai, Naber–Siebers delay, and sparse heteroscedastic GP on RMSE, MAE, and P95 absolute error.

Summary of Recent Experiments

Recent Progress & Evaluations:

- **Traditional Baseline Comparison:** Benchmarked MLP against empirical formula curve fitting (Naber-Siebers, Hiroyasu-Arai).
- **Stability Testing:** Conducted 5-seed Stage-3 evaluation to separate a real effect from seed luck.
- **Machine Learning Baseline:** Implemented sparse heteroscedastic Gaussian Process regression as a non-parametric uncertainty baseline.
- **Feature Engineering Upgrade:** Replaced sparse raw pressure/diameter axes with $A = \Delta P^{0.5} \rho_a^{-0.25} d_n^{0.5}$.
- **OOD Testing:** Added leave-one-nozzle-out evaluation to test unseen nozzle-family generalization.
- **Probabilistic Calibration:** Evaluated reliability, ECE, CRPS, PIT, and Gaussian coverage.

Metric	Meaning
RMSE	Penalizes large point-prediction errors; primary accuracy metric.
MAE	Typical absolute millimetre error; less tail-sensitive than RMSE.
P95 abs. err.	95th percentile of $ \hat{y} - y $; reports tail risk.
Bias	Mean signed error; detects systematic over-/under-prediction.
1σ , 2σ coverage	Fraction with $ y - \mu \leq k\sigma$; calibrated Gaussian targets are 0.683 and 0.954.
ECE	Mean reliability-diagram gap over lower-tail probabilities; lower is better.
CRPS	Proper probabilistic score in mm; lower means better calibration-sharpness trade-off.

Traditional Baseline I: Naber–Siebers Correlation

Formulation: The Naber–Siebers model correlates spray penetration $S(t)$ with physical parameters:

$$S(t) = K \cdot \underbrace{\left(\frac{\Delta P}{\rho_a} \right)^{0.25}}_{\text{Scaling Factor } A} \sqrt{d_n} \cdot \sqrt{\max(t - t_d, 0)}$$

Fitted Parameters:

- K : Global scaling constant.
- t_d : Fitted temporal delay (accounting for hydraulic lag).

Image Processing Data Utilized:

- **CDF Penetration Series:** Cleaned, delay-aligned penetration trajectories from top-view Mie scattering.
- **Cone Angle θ (Optional Variant):** Incorporates spreading factor $\frac{1}{\sqrt{\max(\tan(\theta/2), 10^{-9})}}$ extracted directly from video segmentation.

Traditional Baseline II: Hiroyasu–Arai (1990) & Zhou (2019)

Formulation (Multi-stage):

- **Early Stage** ($t < t_b$): $S = k_v \sqrt{\frac{2\Delta P}{\rho_l}} t$
- **Late Stage** ($t \geq t_b$): $S = k_p \left(\frac{\Delta P}{\rho_a} \right)^{0.25} \sqrt{d_n t}$
- **Breakup Time**: $t_b = k_{bt} \frac{\rho_l d_n}{\sqrt{\rho_a \Delta P}}$
- **Zhou Extension (Post-injection, $t > 2t_i$):**

$$S = \sqrt{2} k_p \left(\frac{\Delta P}{\rho_a} \right)^{0.25} \sqrt{d_n t_i^{0.25}} (t - t_i)^{0.25}$$

Fitted Parameters:

- k_v, k_p, k_{bt} calibrated on the dataset (or literature defaults).

GP Learning Target: Same Regression, Different Prior

Start from the same supervised-learning problem as the MLP

Each pointwise training sample is

$$\mathbf{x}_i = [t, \text{tilt}, \text{plumes}, t_{\text{inj}}, p_{\text{ctrl}}]_i, \quad z_i = \frac{S_i}{A_i}.$$

The physical scaling factor is

$$A_i = (\Delta P_i)^{0.5} \rho_{a,i}^{-0.25} d_{n,i}^{0.5}.$$

The MLP learns $x \mapsto z$. The GP baseline learns the same target and is multiplied by A_i at the end to return to millimetres.

The difference is the function assumption

MLP: represent the function explicitly with parameters, $z_\theta(x)$.

GP: place a probability distribution directly over the unknown function:

$$z_i = f(\mathbf{x}_i) + \epsilon_i.$$

GP Definition: A Distribution over Functions

Finite-dimensional definition

For any finite input set $X = [x_1, \dots, x_n]$, if the function values satisfy

$$\mathbf{f}_X = [f(x_1), \dots, f(x_n)]^\top \sim \mathcal{N}(\mathbf{m}_X, K_{XX}),$$

then

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')).$$

Mean function

$$(\mathbf{m}_X)_i = m(x_i)$$

The implementation uses a constant mean, so the data learn the global offset.

Kernel / covariance

$$(K_{XX})_{ij} = k(x_i, x_j)$$

The kernel says how similar two function values should be.

Kernel Used Here: ARD Matérn-5/2

Distance is scaled by learned length-scales

$$r^2(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^5 \left(\frac{x_j - x'_j}{\ell_j} \right)^2.$$

ℓ_j is the length-scale for input dimension j . A large ℓ_j means the function is insensitive to that feature; a small ℓ_j means small changes in that feature can change the prediction.

Matérn-5/2 covariance

$$k_{\theta}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \sqrt{5}r + \frac{5r^2}{3} \right) \exp(-\sqrt{5}r).$$

It is smooth, but less over-smooth than an RBF kernel, which is useful for penetration traces with transient behaviour.

Code: `ScaleKernel(MaternalKernel(nu=2.5, ard_num_dims=5)).`

Exact GP Posterior: Joint Gaussian Conditioning

Start with the homoscedastic observation model

$$\mathbf{z} = \mathbf{f}_X + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma_n^2 I).$$

For test inputs X_* , the joint distribution is

$$\begin{bmatrix} \mathbf{z} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{m}_X \\ \mathbf{m}_* \end{bmatrix}, \begin{bmatrix} K_{XX} + \sigma_n^2 I & K_{X*} \\ K_{*X} & K_{**} \end{bmatrix} \right).$$

Conditioning gives the prediction formula

$$\begin{aligned} \mu_* &= \mathbf{m}_* + K_{*X}(K_{XX} + \sigma_n^2 I)^{-1}(\mathbf{z} - \mathbf{m}_X), \\ \Sigma_* &= K_{**} - K_{*X}(K_{XX} + \sigma_n^2 I)^{-1}K_{X*}. \end{aligned}$$

This is the mathematical form of “average nearby training evidence and return an uncertainty”.

Why Exact GP Is Not Feasible Here

Hyperparameters are learned by marginal likelihood

Let $K_y = K_{XX} + \sigma_n^2 I$. Then

$$\log p(\mathbf{z}|X, \theta) = -\frac{1}{2}(\mathbf{z} - \mathbf{m})^\top K_y^{-1}(\mathbf{z} - \mathbf{m}) - \frac{1}{2} \log |K_y| - \frac{n}{2} \log 2\pi.$$

The first term rewards fit, the second penalizes overly flexible covariance, and the third is constant.

The scaling bottleneck

Computing $K_y^{-1}\mathbf{z}$ and $\log |K_y|$ usually requires a Cholesky factorization:

$$K_y = LL^\top, \quad \text{cost} = O(n^3), \quad \text{memory} = O(n^2).$$

This task expands trajectories into many time points; per-seed training can reach the million-point scale. Exact GP is therefore not practical.

Sparse GP: Compress the Function with Inducing Points

Core idea

Choose $m \ll n$ inducing inputs

$$Z = [z_1^{(u)}, \dots, z_m^{(u)}], \quad \mathbf{u} = f(Z).$$

They are not labels. They are support locations for the latent function. Training learns the function distribution around these support locations and uses them to explain all n observations.

Conditional structure

The GP prior gives

$$p(\mathbf{f}, \mathbf{u}) = p(\mathbf{f}|\mathbf{u})p(\mathbf{u}), \quad p(\mathbf{u}) = \mathcal{N}(\mathbf{m}_Z, K_{ZZ}).$$

With $m = 256$, the main operations are built around $m \times m$ matrices and minibatch covariance blocks instead of one dense $n \times n$ matrix.

In this implementation, the 256 inducing points are initialized by K-means on the training inputs and then allowed to move during training.

Variational GP: Optimize $q(u)$ Instead of Exact $p(u|D)$

Variational distribution

The exact posterior $p(\mathbf{u}|\mathcal{D})$ is hard to compute, so we use

$$q(\mathbf{u}) = \mathcal{N}(\mathbf{m}_u, S_u),$$

with $\mathbf{m}_u$ and S_u as trainable parameters. For any training function values,

$$q(\mathbf{f}) = \int p(\mathbf{f}|\mathbf{u})q(\mathbf{u}) d\mathbf{u}.$$

ELBO

$$\mathcal{L} = \sum_{i=1}^n \mathbb{E}_{q(f_i)} [\log p(z_i|f_i)] - \text{KL}(q(\mathbf{u}) \parallel p(\mathbf{u})).$$

The first term explains the data; the second prevents $q(u)$ from drifting too far from the GP prior. In minibatch training, the likelihood part is scaled by $n/|\mathcal{B}|$.

The Thesis GP Variant: Two SVGPs in Series

Stage 1: mean SVGP

$$f_{\mu}(x) \sim \mathcal{GP}(m_{\mu}, k_{\mu}), \quad z_i = f_{\mu}(x_i) + \epsilon_i.$$

After training, it gives

$$\hat{\mu}_z(x) = \mathbb{E}[f_{\mu}(x)].$$

Stage 2: log-variance SVGP

First compute residuals from the mean GP:

$$r_i = z_i - \hat{\mu}_z(x_i).$$

Then construct the variance target:

$$g_i = \log(r_i^2 + \varepsilon).$$

A second SVGP learns

$$f_v(x) \approx g, \quad \hat{\sigma}_z^2(x) = \exp(\hat{f}_v(x) + b).$$

b is a validation-set log-variance bias correction.

Returning from Scaled GP Space to Millimetres

Scaled space

The GP is trained and evaluated internally on

$$z = \frac{S}{A_{\text{scale}}},$$

so it predicts

$$\hat{\mu}_z(x), \quad \hat{\sigma}_z(x).$$

Physical space

For penetration in millimetres:

$$\hat{\mu}_S(x) = A_{\text{scale}}(x)\hat{\mu}_z(x), \quad \hat{\sigma}_S(x) = A_{\text{scale}}(x)\hat{\sigma}_z(x).$$

RMSE, MAE, P95, and coverage are all computed in this physical space.

Implementation detail. The headline GP uses residual log-variance as the uncertainty source: `include_mean_posterior_var=false`, so reported σ does not add the mean GP posterior variance.

Mapping the Formulae to the Project Code

Mathematical object	Implementation	Meaning
x	<code>a_only</code> features	time, tilt, plume count, injection duration, control backpressure.
$z = S/A$	<code>target_scaled</code>	Same $\Delta P^{0.5}$ target scaling as the Stage-3 MLP.
k_θ	ARD Matérn-5/2	One learned length-scale per input dimension.
$Z, \mathbf{u}$	256 inducing points	K-means initialization; locations are trainable.
$q(u)$	Cholesky variational distribution	GPyTorch sparse variational posterior.
$\mathcal{L}$	VariationalELBO	Minibatch Adam; mean and logvar each train up to 80 epochs.
$\sigma(x)$	second SVGP on $\log r^2$	Two-stage heteroscedastic uncertainty.

Code entry point: `MLP/MLP_training/run_gp_baseline.py`

Why the GP Baseline Is Fair, but Still Loses on Accuracy

Why it is fair

The GP uses the same splits, the same five seeds, the same A_{scale} target, the same controllable input features, and the same external clean diagnostic set as the MLP comparison.

Where GP is strong

The kernel prior gives a strong smoothness bias, SVGP gives probabilistic predictions, and the second log-variance GP makes uncertainty input-dependent. It is not a weak baseline; it brings coverage close to nominal.

Where the MLP pipeline is stronger

The Stage-3 MLP is not just a generic regressor. It inherits the full fitted trajectory target structure and then refines early-time behaviour against raw CDF observations. The GP learns the same pointwise target family, but it does not include that teacher/raw staged training mechanism.

External Clean Headline Numbers

Model	RMSE	MAE	P95	1σ	2σ
Hiroyasu–Arai calibrated	9.974	7.686	20.493	0.536	0.886
Naber–Siebers delay	8.840	6.780	17.922	0.620	0.893
Sparse heteroscedastic GP, 5-seed mean	6.898	5.206	14.365	0.670	0.950
Stage-3 MLP $\Delta P^{0.5}$, 5-seed mean	6.211	4.587	13.029	0.774	0.981

Relative RMSE reductions for MLP: 37.7% vs HA, 29.7% vs NS, and 10.0% vs GP. The MLP is conservative on clean-set coverage; GP is closer to nominal coverage.

Source: Thesis/generated/baseline_comparison_20260509/headline_comparison.csv

Why Compare against these Baselines?

- **Rigidity of Traditional Models:** Empirical formulas lock the penetration curve into $\sqrt{t}$ or $t^{0.25}$ dependencies, ignoring specific internal nozzle geometry or complex post-injection pressure wave dynamics.
- **Advantage of MLP:** With the **upgraded engineered features** (e.g., $A = \Delta P^{0.5} \rho_a^{-0.25} d_n^{0.5}$), the MLP is anchored to physical dimensions but retains the capacity to learn non-linear residuals.
- **Data Utilization:** Naber-Siebers optional variant relies on the explicit video-measured cone angle θ . The MLP aims to predict penetration mapping directly from controllable condition parameters, without needing "oracle" visual features during inference.